Instrumentum Urbis

Minecraft Cities at scale

Purpose Statement

The purpose of this software would be to create a tool which is capable of generating bulk quantities of unique building assets for Minecraft city building. It would serve as an integral architectural tool for Minecraft, capable of recreating architectural forms using common building techniques, separated by survival-friendly and creative-friendly modes.

Functional Requirements

- Built-in building editor
- Export to Litematica, Axiom, and WorldEdit
- Random asset generator and bulk export
- Regenerate build design at common angles (30, 45, 60)
- Schematic importer

Additional Functions

- Floor plan editor
- Export blueprints to Axiom filepath directly

Design philosophy

This tool aims to be the premier architectural tool for Minecraft, a tool which can enable players to erect beautiful cities with speed never before seen. It needs to have an easy workflow with popular Minecraft building tools such as Axiom. The build editor should be intuitive and simple, but with the power to create great architectural designs.

Compatibility:

This tool should, at minimum, be able to generate builds for Minecraft version 1.21.11 and above. The version the user wants to design their facade for should be one of the first options selected in the editors. The Program should be able to import/export Axiom .bp files as needed.

Build Editor

This could work similarly to a design software like AutoCAD, where features are drawn and then translated to Minecraft at a given scale. This is a lower priority editor type and I would probably only work on this once finishing the Facade Editor.

Facade Editor

As an alternative build editor, this editor would be divided into three “stages”: Base, Body, and Capital. The builder first selects a facade width, where he then can create as many of each stage as he pleases. After completing at least one of each stage, he can move onto another menu which contains various options like “floor range” representing how many floors the buildings will have (minimum is obviously 3, any additional floors will be copies of the body stacked upon each other, this should somehow be represented in-editor), and whether a corner building will have a sharp (90 degree) corner or will be chamfered (additional wall at 45 degree angle). From there, the builder can select how many combinations should be generated using a slider, and maybe a checkbox to generate all possible combinations (with a warning pop up if that value is above some number, to prevent users from crashing their computers). Roofs could be autogenerated, with options such as Gable, Mansard, Convex Mansard, Sloped, etc. The block used for the roof could be determined by the player as an option, additionally with the ability to use custom gradients.

Dev Log 1 - Core App Experience (Oct 12 2026)

While the eventual goal is to have the program run as a mod integrated into the game itself, like Axiom is, I believe that the best choice to start actualizing this idea will be to have it run as a standalone desktop app. I want to start with the Facade editor first and foremost. The core features I want to tackle are the three stages, especially defining the rules governing the stacking of the Body stage when x amount of floors are present. To tackle the different ways a user may construct the walls, I think it's best to define what the “center” of a wall is. Each stage of the facade will have a core “wall” in which ornamentation is stacked in front of (+z) and indents/windows/curtains/etc. are stacked inside of (z=0) or behind (-z). A facade’s wall will have a midpoint where an axis of symmetry may or may not be defined (x=0). The width of the facade will be defined by the user, from then the Core Wall will run *width*/2 in the + and - x directions to give the full length of the wall. The individual floor height (y) will also be defined by the user. The Core Wall will extend *height* in the y direction. Once the Core Wall is generated, the user can customize it, attach ornamentation, add windows, etc. At some point in the future, it would be nice to have a more complex Core Wall generator which can provide the user with a wall that already has windows and perhaps basic ornamentation generated, but that should not be a focus for now. When importing a schematic of any filetype into the editor, the user should first have to define the Core Wall, in its x, y, and z directions. The program should have a check to ensure it was properly defined. For example: a facade should never have a thickness that is greater than its width (|zmax| should never be greater than |xmax|). If the midpoint (x=0) was not defined properly, the program could also adjust the definition so that x=0 is exactly at the middle of the wall’s width.

Upon completing all pieces of the facade, the final facade should be generated with its properly stacked stages, and where all Core Walls align so as to form a signal Core Wall, completely aligned in the proper axes. It is then when the user decides if the facade is allowed to generate as a Corner Building. When defining what a “corner” is, the most basic corner type, the Sharp Corner, will simply be where two perpendicular pieces of the facade intersect so that their Core Walls touch, with exactly one column of blocks in the y direction intersecting. The

corner should be generated so that the inner walls face the inside and the outer walls face the outside. Eventually, a unique type of file format for exporting these facades would be useful, containing key data such as the definitions for the walls, and various generation settings and such, but that doesn't need to be a priority.